AI-DRIVEN MULTI-VENDOR
PREDICTIVE COMMERCE PLATFORM

Complete Project Proposal, End-to-End Workflow &
Corrected Production Architecture

Manager-Ready Technical Document

React.js • Node.js • PostgreSQL • Event-Driven Architecture • Analytics • ML/DL • LLM • WhatsApp

ARCHITECTURAL REVISION

This version preserves the original proposal's 43-section structure and project intent while incorporating the identified architectural corrections. It is intended to be the synchronized architecture baseline for subsequent implementation specifications, database design, API contracts, and phased engineering work.

Authoritative design principle

Transactional systems own facts; events distribute facts; analytics explains them; ML predicts; AI assists users using authorized facts. Search, caches, vector stores and models are derived or computational systems and must not silently become sources of transactional truth.

1. Executive Summary

- The proposed system is a multi-vendor commerce platform in which merchants register, manage stores and listings, maintain inventory, fulfill seller orders, and receive operational intelligence; customers discover products, compare merchant listings, purchase products, manage post-order workflows, and interact through web and WhatsApp.
- The platform begins as a modular monolith with explicit bounded contexts and transactional boundaries. Modules are independently testable and may later be extracted into services only when scale, ownership, or deployment independence justifies the operational cost.
- PostgreSQL is the authoritative transactional source of truth for commerce facts. Redis is limited to cache, rate limiting and selected ephemeral session state. Search indexes, analytics stores, vector stores and ML features are derived representations.
- A transactional outbox publishes versioned domain events after successful database commits. Consumers are idempotent, retryable and observable, with dead-letter and replay mechanisms.
- Marketplace orders explicitly support one customer order containing multiple seller orders, seller-specific fulfillment, partial cancellation/return/refund and settlement.
- Payment is a separate bounded context with payment intents, attempts, provider transactions, verified webhooks, refunds and reconciliation. Payment confirmation never relies solely on a client or conversational message.
- Conversational AI accesses live commerce only through authenticated, authorized, validated tools. The model cannot issue SQL, select arbitrary records, bypass tenant boundaries or invent live price, stock, payment or order facts.

The original proposal's core separation of transactional truth, asynchronous events, analytics, predictive intelligence and conversational AI is retained; the corrections below make the boundaries implementable rather than conceptual.

2. Problem Statement

- Multiple merchants must sell the same canonical product or SKU without duplicating product identity while retaining merchant-specific price, stock, fulfillment and commercial terms.
- Customers require discovery, vendor comparison, availability, checkout, payment, delivery, returns and refunds across potentially multiple sellers in one cart.
- Merchants require tenant-isolated operational visibility, inventory controls, seller-order workflows, settlement visibility and predictive insights.
- Commerce workflows must remain correct under concurrency, retries, duplicate webhooks, provider timeouts and partial failures.
- AI responses about live commerce must be grounded in authoritative backend facts and must respect customer, merchant and administrator authorization.
- The system must support analytics and ML without allowing analytical or AI workloads to compromise transactional correctness.

3. Project Objectives

- Build a complete multi-vendor commerce workflow from merchant onboarding through catalog, listing, inventory, cart, checkout, payment, seller order, fulfillment, return/refund and settlement.
- Define explicit bounded contexts, ownership rules, module dependency rules and transactional boundaries before implementation.
- Design a normalized, transactionally consistent PostgreSQL data model with immutable financial and inventory history.
- Implement secure customer, merchant and administrator access with tenant isolation and object-level authorization.
- Provide search and discovery as derived read models with authoritative transactional filtering.
- Create a reliable event platform using transactional outbox, versioned contracts, idempotent consumers, retries, DLQ and replay.
- Implement analytics and ML with measurable data quality, evaluation, lineage, model versioning and rollback.

- Connect WhatsApp through a secure webhook and conversation gateway, with policy-controlled LLM tools and deterministic fallbacks.
- Demonstrate production system-design concepts including idempotency, concurrency, resilience, observability, CI/CD, backup/restore and disaster recovery.

4. Users and Actors

Actor	Responsibilities
Customer	Browse/search, compare listings, cart, checkout, payment, orders, returns, reviews, preferences and conversational assistance.
Merchant	Registration/verification, store management, listings, prices/offers, inventory, seller orders, fulfillment, returns and merchant analytics.
Administrator	Merchant approval, catalog governance, moderation, disputes, refunds where authorized, platform configuration, security, audit and analytics.
AI/Automation	Interpret requests, retrieve authorized facts, propose or execute explicitly permitted tools, generate grounded responses and trigger eligible workflows.
System/Worker	Process events, notifications, search indexing, analytics ingestion, reconciliation, scheduled jobs and ML pipelines.

5. High-Level Product Scope

Layer	Core capabilities
Core Commerce	Identity, merchants, catalog, variants/SKUs, listings, inventory, cart, checkout, orders, payments, fulfillment, returns and refunds.
Marketplace	Seller orders, commissions, seller earnings, settlement eligibility, payouts and financial ledger.
Discovery	Search, autocomplete, filters, vendor comparison, similar products and progressively enhanced semantic retrieval.
Intelligence	Analytics, dashboards, recommendations, demand forecasting, stockout prediction, propensity and anomaly detection.
Conversational AI	WhatsApp webhook, conversation management, intent understanding, policy-controlled tools, RAG where appropriate and grounded responses.
Platform	Security, observability, CI/CD, resilience, audit, backup/recovery, configuration and feature flags.

6. Complete End-to-End Workflow

```
CUSTOMER / MERCHANT / ADMIN
■
▼
React Web Apps / WhatsApp
■
▼
CDN / WAF / API Gateway
■
▼
Authentication + Authorization + Tenant Context
■
▼
Node.js Modular Monolith
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Identity Merchant Catalog Inventory Commerce
Cart Checkout Order Payment Fulfillment Returns
Promotion Search Notification Customer Admin AI
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■
▼
PostgreSQL — Transactional Truth
■
▼
Transactional Outbox
■
▼
Event Bus / Queue
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Search Analytics Notifications Reconciliation
ML/Features Integrations Background Workers
■
▼
AI Orchestrator
Policy → Authorization → Tool Validation → Authorized Commerce API
■
▼
Grounded Response / Action
```

Service extraction is not assumed. The initial implementation remains a modular monolith with strict module boundaries. Independent services are an evolution option, not a default deployment requirement.

7. Customer Workflow

7.1 Registration and discovery

Customer → Register/Login → Session → Home/Categories/Search → search read model → filters + authorization-aware availability + ranking → product/listing results.

7.2 Product selection

Canonical Product → Variant/SKU → Eligible Merchant Listings → compare price/stock/fulfillment → select listing → add to cart → server-side cart validation.

7.3 Checkout

Cart → Checkout Session → re-price/tax/shipping validation → promotion validation → inventory reservation → seller-order allocation → payment intent → payment provider → verified webhook → order confirmation → reservation allocation/commit → fulfillment → notification.

Failure handling is explicit: payment failure releases reservations; payment success with order-creation uncertainty enters reconciliation rather than charging again; expired checkout sessions release reservations; all transitions are idempotent.

7.4 Order lifecycle

Customer Order: CREATED → PAYMENT_PENDING → PAID → PARTIALLY_FULFILLED / FULFILLED → COMPLETED, with CANCELLED/RETURNED/REFUNDED transitions constrained by policy. Seller Orders independently progress through seller fulfillment states.

7.5 Out-of-stock workflow

Requested listing unavailable → check eligible merchant listings for same SKU → check eligible variants → retrieve similar products → apply customer/business constraints → rank alternatives → present options. Inventory availability is revalidated before purchase.

8. Multi-Vendor Workflow

Merchant Registration → Verification → Store → Canonical Product/SKU Mapping → Merchant Listing → Price/Offer → Inventory Location/Balance → Publish → Customer Discovery → Cart → Checkout → Customer Order → Seller Order Split → Seller Fulfillment → Shipment → Delivery → Review/Return → Settlement.

A canonical product is distinct from a merchant listing. A listing represents a seller-specific offer for a purchasable SKU and is the commercial/inventory boundary for marketplace stock.

A single customer order may contain multiple seller orders. Each seller order owns seller-specific items, fulfillment, shipment, cancellation eligibility, return handling and settlement calculations.

9. Product and Catalog Model

```
Category
■■■ Subcategory
■■■ Product
■■■ Variant / SKU
■■■ Merchant Listing A
■■■ Merchant Listing B
■■■ Merchant Listing C
```

Categories provide taxonomy. Products provide canonical identity. Variants/SKUs represent purchasable configurations. Merchant listings represent seller-specific commercial offers. Attributes support filtering and search. Catalog governance controls duplicates, incomplete data, moderation and publication state.

Listing publication is separate from product approval. A seller cannot publish an unapproved or commercially invalid listing merely because the canonical product exists.

10. Inventory Architecture

```
Inventory is associated with a sellable merchant listing/SKU and, where applicable, an inventory location.

Available = On Hand - Reserved - NonSellable
On Hand → Reserved → Allocated → Picked → Packed → Shipped → Delivered
Reserved → Released
Returned → Inspection → Restocked / Damaged
```

Maintain an immutable inventory movement ledger. Use database transactions with row/version locking or equivalent atomic conditional updates for reservations. Reservation expiry is mandatory for abandoned checkout sessions.

Inventory states are explicit: AVAILABLE, RESERVED, ALLOCATED, PICKED, PACKED, SHIPPED, DELIVERED, RELEASED, RETURNED, INSPECTION and DAMAGED where applicable.

Redis and LLMs are never inventory truth. Reconciliation compares derived balances with authoritative movements and flags discrepancies without silently overwriting history.

11. Database Architecture

Bounded context	Representative entities
Identity & Access	users, roles, permissions, user_roles, sessions, refresh_tokens, authentication_events
Merchant	merchants, stores, merchant_users, verification_records, merchant_settings
Catalog	categories, products, variants, attributes, brands, catalog_approvals
Marketplace	merchant_listings, listing_prices, listing_offers, listing_publications
Inventory	inventory_locations, inventory_balances, inventory_reservations, inventory_movements, stock_adjustments
Cart & Checkout	carts, cart_items, checkout_sessions, checkout_items, checkout_attempts
Orders	customer_orders, seller_orders, order_items, order_status_history, seller_order_status_history
Fulfillment	shipments, shipment_items, carriers, tracking_events, delivery_events
Payment	payment_intents, payment_attempts, payment_transactions, payment_webhooks, refunds, reconciliation_records
Settlement	seller_accounts, commission_rules, seller_earnings, settlement_batches, payouts, payout_transactions, financial_ledger
Customer	addresses, preferences, wishlists, reviews, consents
Promotion & Pricing	offers, coupons, campaigns, promotion_rules, eligibility, pricing_snapshots, tax_snapshots, shipping_fee_snapshots
Events	outbox_events, event_consumption, webhook_deliveries, event_replay_records
Search	index_jobs, index_versions, search_audit where required; search engine remains derived
AI	conversations, messages, tool_calls, ai_runs, ai_feedback, policy_decisions
ML & Analytics	features, datasets, model_versions, predictions, candidates, outcomes, fact/dimension stores
Security & Audit	audit_logs, login_events, security_events, access_decisions

12. Database Engineering Concepts

- Normalization and controlled denormalization with explicit ownership.
- Primary/foreign keys, unique/check/not-null constraints and state-validating constraints.
- Composite, partial, covering and expression indexes based on measured query patterns.
- Transactions and appropriate isolation levels; optimistic versioning and row locking for contention points.
- Immutable order financial snapshots, inventory movements, payment provider references and audit history.
- Soft deletion only where historical references must survive; never use deletion to rewrite financial truth.
- Partition high-volume event/audit tables only after workload evidence.
- Use read replicas or analytical stores for reporting rather than burdening OLTP.
- All schema changes use version-controlled migrations and tested rollback/forward migration procedures.
- Tenant keys are explicit on tenant-scoped data and are enforced at application boundaries and, where appropriate, PostgreSQL Row-Level Security.

13. Transaction and Idempotency Design

Operation	Protection
Checkout	Idempotency key + checkout state machine + transactional validation.
Inventory reservation	Atomic availability check + reservation row + expiry + idempotency.
Payment callback	Provider signature verification + provider event ID + idempotent state transition.
Webhook	Verified source + replay protection + durable event/message ID.
Order cancellation	State-machine validation + compensating inventory/payment actions.
Order creation	Unique checkout/idempotency reference; never create a second order for the same successful checkout.
Settlement	Immutable ledger entries + settlement batch idempotency + provider reference.
Event publishing	Transactional outbox; publish after commit; consumers remain idempotent.

14. Search and Discovery

Customer Query → query understanding → lexical retrieval + semantic retrieval → candidate merge → authoritative filters → availability/business rules → ranking → results → search event.

PostgreSQL remains authoritative for product/listing state. Search engines are derived read models updated from committed changes/events. Search index lag must not be treated as a license to report stale transactional stock or price.

Begin with PostgreSQL search where appropriate. Introduce OpenSearch/Elasticsearch when measured relevance, scale or latency requires it. Support autocomplete, typo tolerance, synonyms, facets and semantic retrieval progressively.

Indexing pipeline: Listing/Product Changed → outbox event → index job → versioned document → search engine. Failed indexing retries and enters a DLQ; reindexing is replayable.

15. Recommendation Architecture

Behavior/catalog data → candidate generation → hard eligibility filters → ranking → diversity/business rules → recommendation → impression/click/cart/purchase/ignore → feedback dataset → evaluation → next model version.

Candidate sources include same-SKU alternatives, similar products/variants, frequently bought together, popularity, personalized history, trending products, price-sensitive alternatives and wishlist/recently viewed items.

Eligibility must exclude unavailable, unauthorized, unpublished, restricted or otherwise invalid listings before ranking. Recommendation is never allowed to bypass inventory or commerce authorization.

16. Predictive Analytics

Use case	Input signals	Output/action
Demand forecasting	Historical sales, seasonality, price, promotion, availability	Forecast → replenishment planning
Stockout prediction	Inventory, reservations, sales velocity, forecast	Risk/time-to-stockout → alert
Purchase propensity	Search, views, cart, purchase history	Intent score → personalization
Customer inactivity	Purchase intervals, engagement	Risk segment → eligible retention workflow
Offer response	Price/discount and historical response	Expected response → promotion analysis
Trend detection	Search/view/purchase velocity	Trend signal → merchant insight
Anomaly detection	Orders, payments, inventory and operational metrics	Anomaly score → review/alert

17. Predictive Customer Engagement Workflow

Customer searches/views product → behavior event → feature generation → propensity/recommendation model → notification eligibility → consent + channel rules + frequency cap + quiet period + business rules → WhatsApp/email/push template → delivery/click/purchase outcome → analytics/ML feedback.

A search event alone must not automatically trigger a message. Every outbound workflow must verify consent, channel eligibility, suppression rules, frequency caps, business purpose and provider requirements.

18. Analytics Architecture

Web/API/Merchant/WhatsApp events → event validation → durable event stream → analytical ingestion → facts + dimensions → dashboards and governed feature datasets.

Example facts: fact_orders, fact_order_items, fact_product_views, fact_searches, fact_cart_events, fact_payments, fact_inventory_movements, fact_recommendation_impressions. Example dimensions: dim_customer, dim_product, dim_variant, dim_merchant, dim_category, dim_date, dim_location.

Separate OLTP from heavy analytics. Track freshness, completeness, uniqueness, validity, consistency and lineage. Version event schemas and maintain source-to-feature lineage for critical ML inputs.

19. LLM + WhatsApp Architecture

WhatsApp Provider → verified webhook → message normalization → tenant/channel resolution → authenticated conversation context → policy and intent layer → LLM → tool policy/authorization → argument validation → authorized backend tool → authoritative result → result validation → LLM response → output guardrails → WhatsApp response → delivery/outcome event.

The selected transactional backend remains Node.js + PostgreSQL. MongoDB is not part of the chosen platform architecture. Any WhatsApp reference implementation must use the same transactional source of truth rather than introducing a competing order/catalog database.

WhatsApp is a channel adapter. Conversation state may use Redis for short-lived ephemeral state, but durable conversations, messages, tool calls, audit information and commerce references are persisted in PostgreSQL as required by retention policy.

19.1 WhatsApp Business Platform Fundamentals

- WhatsApp Business Account and phone number identify the channel tenant.
- Webhook receives inbound messages, interactive selections and delivery/status events.
- Catalog data is served from the authoritative commerce backend; chat is not a second catalog.
- Flow/conversation state is tenant-scoped and persisted appropriately.

- Orders are immutable commercial snapshots linked to the transactional order system.
- Provider constraints such as message formats, template requirements, delivery windows and payload limits are configuration/adapter concerns and must not leak into core domain logic.

19.2 Channel Payload and Interaction Rules

- Interactive controls use stable IDs, not merchant-editable display labels.
- Lists and buttons are paginated and validated against provider constraints before sending.
- Every interactive path has a text fallback and an explicit error/recovery message.
- Outbound messages outside provider-defined customer-care windows use approved templates where required.
- Media uses provider-reachable URLs or approved media upload flows; inaccessible media falls back safely to text.
- Provider webhook verification, replay protection and durable event IDs are mandatory.

19.3 Catalog Data Model for Conversational Commerce

- Chat reads canonical categories/products and eligible merchant listings through backend APIs.
- Price and stock shown in conversation are fetched from current authorized commerce tools; cached values are labeled/validated according to their freshness policy.
- Portion/variant choices map to real SKU/listing identifiers.
- Conversation product availability is not a substitute for checkout-time inventory reservation.

19.4 Conversational Commerce Flow

- Browse → Product → Variant/Listing → Cart → Checkout Session → payment/order workflow.
- Empty results, unavailable products, invalid inputs and failed dependencies produce recoverable responses rather than dead ends.
- Global commands such as STATUS, HELP and CANCEL are intercepted by a controlled command layer and authorization is rechecked before action.

19.5 Payments over WhatsApp

Where a payment gateway is not yet integrated, a UPI intent link may be used only as an explicitly lower-assurance/manual verification workflow. The chat layer does not move money and does not mark payment as successful merely because the customer says it was paid.

For production automated payments, the payment provider callback/webhook is the source of payment confirmation after signature verification, event deduplication and state validation. Manual verification is a transitional operating mode, not the architectural definition of payment truth.

19.6 Order Lifecycle in Chat Commerce

The chat references the core Customer Order/Seller Order model. It does not create an independent order schema that can diverge from web checkout.

Payment and order status are separate state machines. Customer-visible status is derived from authoritative states and permission-filtered data.

19.7 Inventory Model

A lightweight availability flag may exist as a presentation optimization, but production checkout uses authoritative listing/SKU inventory balances and atomic reservations. A boolean availability flag cannot guarantee stock correctness under concurrency.

All actual stock-affecting operations use the inventory bounded context and ledger.

19.8 Multi-Tenancy, Sessions and Webhook Reliability

- Resolve tenant from verified channel identity and carry tenant context through every application/repository operation.
- Reject or fail closed on missing tenant context for tenant-scoped operations.
- Redis may hold short-lived session state, but durable message/event IDs and important conversation records survive restarts.
- Duplicate inbound deliveries are deduplicated by durable provider message/event ID.

- Delivery/read status events update message telemetry only and never trigger conversational business actions.

19.9 Operational Field Guide and Go-Live Checklist

- Webhook is subscribed and signature/verification checks are tested against the live endpoint.
- Secrets/tokens are stored outside source control and rotated according to policy.
- Interactive payloads are validated against provider constraints and paginated.
- Merchant settings, payment configuration, serviceability, templates and notification preferences are complete.
- Payment, order, inventory and notification failure paths are tested end-to-end.
- Support and escalation paths exist inside the channel.
- Provider outage, duplicate delivery, stale session and webhook replay tests have passed.

19.10 Example

```
Customer: “Is iPhone 15 available under my budget?” → LLM identifies intent/constraints →
search_products(query, filters) → backend retrieves eligible listings → inventory/price validation → LLM
explains current options → grounded response.
```

19.11 Tool Examples

- search_products(query, filters)
- get_product(product_id)
- compare_vendor_listings(product_id)
- check_inventory(listing_id)
- find_similar_products(product_id, constraints)
- get_order(order_id)
- subscribe_alert(customer_id, product_id, alert_type)

20. LLM Security and Guardrails

- Never provide arbitrary SQL, arbitrary database access, filesystem access or unrestricted internal APIs to the LLM.
- Every tool enforces authentication, tenant authorization, object-level authorization and schema validation independently of model output.
- Introduce a tool policy layer between model selection and execution.
- Classify tools as read-only, reversible and sensitive/destructive. Sensitive actions require stronger authorization and explicit confirmation where appropriate.
- Validate tool arguments, enforce allowlists and validate tool results before generation.
- Protect against prompt injection and instruction conflicts.
- Never infer authorization from conversation text alone.
- Do not allow the model to invent price, stock, payment, settlement or order state.
- Log tool calls/outcomes for audit and evaluation while minimizing sensitive data retention.
- Provide deterministic fallback when the LLM or model provider is unavailable.

```
AI TOOL EXECUTION
LLM proposal
↓
Tool Policy
↓
Authentication
↓
Tenant/Object Authorization
↓
Argument Validation
↓
Backend Tool
↓
Authoritative Result
↓
Result Validation
↓
LLM / Deterministic Response
```

21. RAG Architecture

Question → classify live transactional request vs knowledge request.

Knowledge request → approved documents → parse/chunk → metadata/effective dates → embedding/index → retrieval → reranking → source-aware generation.

Live commerce request → authorized backend tool → authoritative result.

Mixed request → policy-controlled document retrieval + live commerce tool → grounded response.

Vector retrieval never becomes the authoritative source for current price, inventory, payment, order status, settlement or other mutable transactional facts.

- Version documents and track source identifiers/effective dates.
- Protect documents and retrieval against prompt injection.
- Measure retrieval quality separately from generation quality.
- Store citations/source references where the user experience and governance require them.

22. Security Architecture

Security domain	Controls
Authentication	Secure password hashing, secure sessions/tokens, refresh rotation, MFA for privileged roles, OTP where appropriate, session revocation.
Authorization	RBAC plus object-level authorization, tenant context, seller/customer ownership checks and deny-by-default policy.
Tenant isolation	Application-layer tenant enforcement plus database/RLS defense in depth where appropriate.
API security	Validation, rate limits, timeouts, pagination/request limits, idempotency and standardized errors.
Web security	CSP, XSS protections, secure cookies, CSRF protection where applicable and secure headers.
Database	Parameterized queries, least privilege, TLS, restricted network access, encrypted backups and controlled migrations.
Webhook	Signature verification, replay protection, provider event ID deduplication and payload validation.
Payment	Provider-hosted/tokenized payment where appropriate; minimize sensitive payment data storage.
AI	Tool authorization, prompt-injection defenses, output validation, data minimization and tenant isolation.
Secrets	Secret manager, rotation, short-lived credentials where possible; no secrets in repository or logs.
Audit	Privileged actions, access decisions, configuration changes, payment/admin interventions and security events.
Privacy	Consent, preferences, retention, access/deletion processes and purpose limitation as applicable.

23. Threat Model

- Broken access control / IDOR / BOLA.
- Merchant-to-merchant data leakage.
- SQL injection and malicious input.
- Credential stuffing, token theft and session abuse.

- Webhook spoofing, replay and duplicate delivery.
- Inventory race conditions and overselling.
- Coupon, promotion, refund and settlement abuse.
- Order/payment tampering and reconciliation errors.
- Malicious file upload.
- Prompt injection, tool abuse and unauthorized AI actions.
- Sensitive information leakage through logs, prompts or analytics.
- Denial-of-service and AI-cost abuse.
- Third-party/provider and dependency/supply-chain vulnerabilities.
- Financial ledger inconsistency or duplicate settlement.

Threat modelling maps assets, trust boundaries, attack paths, mitigations and verification tests. Security controls are accepted only after automated/manual verification, not merely because they appear in the architecture document.

24. Event-Driven Architecture

Event	Typical consumers
commerce.order.created.v1	Inventory, fulfillment, analytics, notification, seller-order workflows
commerce.payment.confirmed.v1	Order confirmation, fulfillment, notification, analytics
inventory.stock.updated.v1	Search indexing, alerts, analytics, recommendations
inventory.stock.low.v1	Merchant alerts, forecasting, replenishment workflows
catalog.listing.changed.v1	Search indexing, recommendation candidates, analytics
commerce.cart.item_added.v1	Analytics, eligible abandonment workflows
customer.product.viewed.v1	Analytics, recommendation candidates
commerce.return.requested.v1	Returns workflow, seller operations, analytics
settlement.earning.created.v1	Settlement, merchant analytics, finance
whatsapp.message.received.v1	Conversation/AI channel
ml.prediction.created.v1	Analytics, dashboards, eligible business workflows

Standard event envelope: event_id, event_type, event_version, occurred_at, producer, tenant_id, actor_id where applicable, aggregate_type, aggregate_id, correlation_id, causation_id, schema_version and payload.

Event names use a single versioned convention such as commerce.order.created.v1. Producers own event contracts; consumers validate versions and must remain idempotent.

25. Event Reliability

- Transactional outbox connects committed database state to event publication.
- At-least-once delivery is assumed; exactly-once business effects are achieved through idempotent consumers and unique business references.
- Retries use exponential backoff and bounded attempts.
- Dead-letter queues capture poison messages with reason metadata.
- Event schema versioning prevents silent contract breakage.
- Correlation and causation IDs support distributed tracing and workflow reconstruction.
- Consumer lag, retry counts and DLQ depth are monitored.
- Replay is supported from durable event records where business semantics permit.
- Event consumers must not mutate source-of-truth facts without passing through the owning bounded context.

26. Caching and Performance

- Use Redis for cache, rate limiting and selected ephemeral session state, never as authoritative inventory/payment/order truth.
- Use cache-aside with explicit TTL, invalidation and stale-read policy.
- Prevent cache stampedes with jitter/locking where justified.
- Use CDN/object storage for media.
- Measure hit ratio, stale-read tolerance, eviction and dependency latency.
- Use query plans and measured workload evidence before adding indexes or caches.
- Define performance budgets for API, checkout, search and AI tools.
- Do not optimize by weakening correctness of transactional state.

27. Failure and Recovery Workflows

Failure	Expected behavior
LLM unavailable	Fallback to deterministic search/order workflows or a safe unavailable response.
ML recommendation unavailable	Popularity/category/content fallback.
Search index stale	Use transactional backend for critical validation; communicate/handle stale discovery safely.
Queue consumer failure	Retry → DLQ → alert → operator/replay.
Payment timeout	Persist attempt, reconcile by provider reference, do not duplicate charge/order.
Payment success but order uncertain	Enter reconciliation workflow; do not create a second charge.
Inventory conflict	Atomic rejection/adjustment; never confirm unavailable quantity.
External API slow	Timeout + circuit breaker + fallback.
WhatsApp provider outage	Queue eligible notifications/retries and expose delivery status.
Bad model release	Evaluation gate + model version rollback.
Database outage	Fail safely for writes; use HA/backup/restore strategy.
Settlement provider failure	Preserve ledger state; retry payout without duplicating financial entries.

28. Distributed-System Concepts

- Timeouts and retries with bounded retry budgets.
- Circuit breakers and bulkheads for external dependencies.
- Rate limiting and backpressure.
- Idempotency and deduplication.
- Transactional outbox.
- Saga/compensating actions for multi-step workflows.
- Eventual consistency only where explicitly accepted by the owning domain.
- Distributed tracing.
- Graceful degradation.
- Reconciliation jobs for financial, inventory and integration boundaries.

29. ML and Deep Learning Strategy

Problem	Baseline	Advanced
Demand forecast	Moving average / exponential smoothing	Gradient boosting or neural time-series where justified
Purchase propensity	Logistic/tree model	Sequence/deep model where justified
Recommendation	Popularity / collaborative filtering	Two-tower retrieval / neural ranking
Product similarity	Structured attributes/rules	Transformer embeddings
Intent classification	Rules/classifier	Transformer/LLM
Catalog understanding	Structured metadata	Multimodal/vision embeddings if needed

The project uses baseline-first engineering. A complex model is promoted only when offline and, where safe, online evaluation demonstrates meaningful improvement against a simpler baseline.

MLOps lifecycle: dataset version → feature generation → training → evaluation → model registry → approval → deployment → monitoring → drift detection → retraining/rollback.

30. AI/ML Evaluation

Component	Example metrics
Intent	Accuracy, precision, recall, F1 where appropriate.
Tool calling	Valid-tool rate, argument correctness, authorization rejection rate, execution success.
Grounding	Supported-answer rate, source correctness, hallucination/error rate.
Recommendations	CTR, add-to-cart, conversion, coverage, diversity.
Forecasting	MAE, RMSE, WAPE or context-appropriate metric.
Classification/prediction	Precision, recall, ROC-AUC/PR-AUC, calibration where appropriate.
Operations	Latency, cost, error, escalation and fallback rate.
ML operations	Drift, data freshness, feature quality, model version health.

- Version models, prompts, retrieval configuration and datasets.
- Maintain an evaluation suite before changing production AI behavior.
- Use human review for ambiguous/high-impact cases.
- Monitor feature/model drift and prediction quality over time.
- Do not compare models on a different population/time period without explicitly documenting the difference.

31. Observability and SRE

Signal	Examples
Metrics	RPS, p95/p99 latency, error rate, DB latency, queue lag, cache hit rate, reservation conflicts, payment success rate.
Logs	Structured logs with request_id, trace_id, correlation_id, tenant_id, actor_id and event_id, with sensitive-data redaction.
Traces	API → module → database → queue → provider → AI tool.
Business telemetry	Conversion, stockout, order success, payment reconciliation, seller fulfillment and recommendation outcomes.
AI telemetry	Intent, tool success, grounding, latency, fallback and token/cost usage.
Alerts	Availability, latency, error spikes, queue backlog, payment/reconciliation anomalies and security anomalies.

- Define SLIs/SLOs for critical APIs, checkout, payment callbacks, search and notification delivery.
- Maintain health/readiness checks and operational runbooks.
- Perform backup restore drills.
- Define business-approved RTO/RPO targets before production launch; do not invent values in architecture without owner approval.
- Use feature flags and controlled rollback.

32. DevOps and CI/CD

```
Developer → Pull Request → lint/typecheck → unit tests → integration tests → SAST → dependency scan → secret scan → container scan → build → staging → smoke/E2E/DAST as applicable → approval → production → monitor release health → rollback if required.
```

Containerized builds and reproducible artifacts. Secrets outside source control. Versioned database migrations. Infrastructure as code where appropriate. Release traceability and rollback procedures.

Production promotion is gated by automated quality/security checks and explicit environment configuration validation.

33. Testing Strategy

Test layer	Examples
Unit	Pricing, tax abstraction, promotion rules, validation, state transitions, domain rules, inventory calculations.
Integration	PostgreSQL repositories, transactions, outbox, queues, payment adapters, search indexing.
API	Authentication, authorization, tenant isolation, pagination, idempotency, standardized errors.
E2E	Search → listing → cart → checkout → payment → customer order → seller order → fulfillment.
Concurrency	Simultaneous reservation/checkout, duplicate payment callbacks, duplicate webhook delivery.
Security	IDOR/BOLA, injection, auth abuse, webhook replay, prompt injection, tenant leakage.
Performance	Load, stress, soak, queue backlog and dependency latency.
AI	Intent, tool selection, authorization rejection, grounding, prompt injection, fallback.
ML	Offline evaluation, drift, calibration, feature quality and rollback.
Resilience	Dependency failures, retries, circuit breakers, queue outages, database failover and recovery.

34. System Design Trade-offs

Decision	Trade-off to document
Modular monolith vs microservices	Simplicity/operability vs independent scaling/deployment. Initial default is modular monolith.
PostgreSQL vs additional datastore	Strong transactional model vs specialized access patterns.
Redis	Latency and scale vs consistency/staleness complexity.
Search engine	Relevance/scale vs operational complexity; derived read model only.
Real-time vs batch ML	Freshness vs cost/complexity.
RAG vs fine-tuning	Knowledge freshness vs training/maintenance cost.
LLM provider/model	Quality vs latency, cost, privacy and provider dependency.
Deep model vs baseline	Complexity vs measured improvement.
Strong vs eventual consistency	Correctness vs availability/throughput for each explicit domain.
Build vs managed service	Control vs engineering/operational effort.
Manual vs automated payment verification	Operational simplicity vs assurance/scale; manual verification is transitional.
Ledger vs mutable balance	Auditability and reconciliation vs implementation simplicity; immutable ledger is authoritative for financial history.

Architecture decisions are recorded as ADRs with context, options, decision, consequences, rejected alternatives and rollback/exit criteria.

35. Technology Stack and Responsibility

Technology	Role
React.js	Customer, merchant and admin web interfaces.
Node.js	Backend APIs, bounded-context application/domain logic and integrations.
PostgreSQL	Transactional source of truth and durable domain/audit records.
Redis	Cache, rate limiting and selected ephemeral session state.
Queue/Event Bus	Asynchronous event processing, workers and integration decoupling.
Search Engine (as needed)	High-scale/faceted/semantic discovery as a derived read model.
Object Storage/CDN	Product/media assets and distribution.
LLM	Natural-language understanding and response generation behind policy/tool boundaries.
Embeddings/vector retrieval	Semantic similarity and RAG only.
ML/DL stack	Forecasting, ranking, prediction and anomaly detection with MLOps controls.
Docker/CI-CD	Reproducible builds, deployment and release controls.
Observability stack	Metrics, logs, traces, alerts and business telemetry.

36. Why This Architecture Is Production-Oriented

- Business-critical facts remain in authoritative transactional systems.
- Asynchronous workloads do not block core checkout unnecessarily.
- AI is replaceable and cannot own business truth.
- Search and analytics evolve independently as derived systems.
- ML models are versioned and evaluated rather than treated as opaque logic.
- Marketplace financial flows have explicit seller-order and settlement boundaries.
- Failures have explicit fallback, reconciliation or compensation behavior.
- Security is enforced at authentication, authorization, tenant, tool and data boundaries.
- Observability covers technical and business health.
- The architecture can start as a modular monolith and evolve where justified.

37. Implementation Roadmap

Phase	Scope	Exit condition
1 — Architecture & Repository Foundation	Monorepo/module conventions, environments, CI, coding standards, ADR process, base observability and configuration.	Build/test pipeline works; module dependency rules enforced.
2 — Identity & Merchant	Authentication, sessions, RBAC, tenant context, merchant onboarding and verification.	Authorization/tenant tests pass.
3 — Catalog & Listings	Categories, products, variants, canonical identity, merchant listings, media and publication.	Catalog/listing invariants enforced.
4 — Inventory	Inventory locations, balances, ledger, reservations, concurrency and reconciliation.	Concurrent reservation tests cannot oversell.
5 — Cart & Checkout	Cart, checkout session, pricing/promotion/tax/shipping abstraction, idempotency.	Checkout state machine and failure compensation pass.
6 — Orders & Payments	Customer/seller orders, payment intents/attempts/transactions, webhooks, refunds, reconciliation.	Payment/order reconciliation is idempotent.
7 — Fulfillment & Returns	Shipments, tracking, delivery, return workflow and refund linkage.	Partial seller fulfillment/returns work.
8 — Marketplace Finance	Commission, seller earnings, financial ledger, settlement eligibility and payouts.	Settlement is auditable and idempotent.
9 — Search	Indexing, filters, autocomplete, relevance and derived read model.	Search rebuild/replay works.
10 — Events & Notifications	Outbox, event bus, workers, DLQ, replay, notification module and templates.	Event reliability/observability tests pass.
11 — Analytics	Facts, dimensions, ingestion, quality checks and dashboards.	Data freshness/quality monitored.
12 — Recommendations	Candidate generation, eligibility, ranking, feedback and evaluation.	Baseline measurable and reproducible.
13 — Predictive ML	Forecasting, stockout, propensity, anomaly detection and MLOps.	Models versioned/evaluated/rollback-capable.
14 — WhatsApp AI	Verified webhook, conversation gateway, policy, tools, LLM, RAG and guardrails.	AI cannot bypass authorization; live facts grounded.
15 — Security & Production Hardening	Threat model, security tests, load/resilience tests, observability, backup/restore and DR.	Release gates and recovery drills pass.
16 — Advanced Intelligence	Semantic search, advanced ranking, deep learning and model monitoring where justified.	Advanced capability demonstrates measurable benefit.
17 — Controlled Service Extraction	Extract only modules with proven scaling/ownership/deployment need.	ADR, contract tests, observability and rollback exist for each extraction.

38. Proposed Project Deliverables

- PRD and business requirements.
- Complete SRS.
- Bounded-context and module dependency specification.

- End-to-end workflow diagrams and state machines.
- ER diagram and database design.
- UML use-case, activity, sequence, class, component and deployment diagrams.
- C4 architecture diagrams.
- API/OpenAPI specification and error/idempotency conventions.
- Event contracts and event-flow documentation.
- Security architecture, tenant-isolation model and threat model.
- Payment, fulfillment, return/refund and marketplace settlement specifications.
- AI/LLM tool authorization and RAG specification.
- ML experiment/evaluation and MLOps report.
- Test strategy and reports.
- Deployment and CI/CD architecture.
- Observability/SLO documentation.
- Disaster recovery and operational runbooks.
- ADR set for major architectural decisions.

39. Definition of Done

- Core business workflows have documented state machines.
- Bounded contexts have explicit ownership and prohibited direct dependencies.
- Database constraints prevent invalid core states.
- Concurrent checkout cannot oversell tested inventory.
- Retries cannot create duplicate orders, payments, payouts or refunds.
- External webhooks are verified, durable and idempotent.
- Customer orders can split into seller orders with independent fulfillment and financial attribution.
- Seller earnings and settlement are traceable to immutable financial records.
- AI cannot execute arbitrary database operations.
- Live AI commerce answers are grounded in authorized tool results.
- Events have versioned schemas, quality checks and replay strategy.
- Security threats have mitigations and automated/manual tests.
- Logs, metrics and traces provide operational visibility with sensitive-data redaction.
- Backups have been restored successfully in a test.
- Production deployment supports safe rollback.
- Payment and inventory reconciliation procedures have been tested.
- Major architecture decisions are documented as ADRs.
- Open production decisions are explicitly recorded rather than silently assumed.

40. Final Project Workflow — Manager View

```
MERCHANT SIDE
Merchant → Registration/Verification → Store
→ Canonical Product/SKU → Merchant Listing
→ Price/Offer → Inventory Location/Balance → Publish
→ Customer Order → Seller Order → Fulfillment
→ Shipment → Delivery → Returns/Adjustments
→ Earnings → Settlement → Analytics/Predictions
```

```
CUSTOMER SIDE
Customer → Login → Search/Browse → Filters/Ranking
→ Product/Variant → Compare Vendors → Cart
→ Checkout Session → Price/Tax/Shipping/Promotion Validation
→ Inventory Reservation → Payment Intent
→ Verified Payment → Customer Order
→ Seller Order(s) → Fulfillment → Review/Return/Refund
```

```
INTELLIGENCE SIDE
Customer/Merchant Events → Transactional Outbox
```

```
→ Event Bus → Analytics
→ Recommendations / Forecasting / Propensity / Anomaly Detection
→ Business Rules → Personalization / Alerts / Merchant Insights
```

```
WHATSAPP AI SIDE
WhatsApp → Verified Webhook → Tenant Resolution
→ Conversation/Intent → Policy
→ LLM Tool Proposal → Authorization → Authorized Backend Tool
→ Live Product/Inventory/Order Data
→ Result Validation → Grounded Response
→ Outcome Event → Analytics/ML Feedback
```

```
SECURITY/PLATFORM SIDE
Every layer → Authentication → Authorization → Tenant Isolation
→ Validation → Rate Limiting → Audit → Encryption/Secrets
→ Monitoring → Failure Recovery → Backup/DR
→ Continuous Security Testing
```

41. Expected Business and Technical Outcomes

- Customers receive richer product discovery, transparent merchant comparison and conversational assistance.
- Merchants receive operational visibility, seller-order controls, settlement visibility and predictive inventory insights.
- Administrators receive governance, security and platform analytics.
- The organization obtains reusable APIs and versioned event contracts for future integrations.
- The architecture provides a foundation for additional channels, vendors and AI capabilities.
- Core commerce remains correct and testable even when search, AI, analytics, ML or third-party providers fail.

42. Scope Governance

Priority	Scope
Must-have foundation	Identity, merchant, catalog, listings, inventory, cart, checkout, orders, payments, fulfillment, returns, security and tenant isolation.
Marketplace core	Seller-order splitting, commissions, ledger, settlement and payouts.
Core intelligence	Events, analytics, search, recommendations and merchant dashboards.
Advanced intelligence	Forecasting, propensity, anomaly detection and semantic search.
Conversational AI	WhatsApp, LLM tools, RAG, guardrails and AI evaluation.
Production hardening	Observability, resilience, load testing, DR, security testing, backup/restore and release controls.
Future expansion	Advanced deep learning, multimodal catalog and additional channels after measurable business/technical justification.

The core commerce system must become correct, observable and testable before advanced AI is introduced. Advanced components are added only after underlying data, event contracts and security boundaries are stable.

43. Conclusion

This project is designed as an integrated commerce and intelligence platform rather than a conventional e-commerce application. The architecture separates transactional truth, asynchronous events, analytics, predictive intelligence and conversational AI while providing explicit secure interfaces between them.

The corrected architecture adds the implementation boundaries required for a real multi-vendor system: explicit bounded contexts, modular-monolith ownership, multi-seller order splitting, inventory and payment state machines, seller settlement, versioned event contracts, tenant isolation, search-as-derived-read-model, AI tool authorization, analytics/ML governance, and operational reconciliation.

The objective is not to use the maximum number of technologies; it is to use each architectural capability where it solves a real system problem while preserving correctness, auditability, security and operability.

This document should serve as the synchronized architecture baseline. Detailed database DDL, OpenAPI contracts, event schemas, repository structure, environment configuration, deployment manifests and phase-specific implementation prompts should be generated from this baseline and must not contradict it.

- PostgreSQL owns transactional commerce facts.
- Inventory truth belongs to the Inventory bounded context; payment truth belongs to the Payment bounded context; financial settlement truth belongs to the Settlement/Ledger bounded context.
- Canonical Product is not the same thing as Merchant Listing.
- Customer Order may contain multiple Seller Orders.
- Search, Redis, analytics stores, vector stores and ML predictions are derived systems.
- All external callbacks/webhooks are authenticated, verified and idempotent.
- All tenant-scoped access carries verified tenant context.
- AI never receives arbitrary SQL or unrestricted backend access.
- AI tools enforce authorization independently of the model.
- Sensitive AI actions require explicit policy and confirmation where applicable.
- Every domain event has a versioned envelope and stable event ID.
- Consumers assume at-least-once delivery and implement idempotency.
- Financial and inventory history is append-oriented and auditable.
- State transitions are validated by the owning domain, not by clients or LLMs.
- Service extraction requires an ADR and must preserve API/event/observability contracts.
- Business-approved RTO/RPO targets.
- Final payment provider(s) and provider-specific production configuration.
- Final tax jurisdiction/rules and compliance configuration.
- Production infrastructure sizing, regions and availability topology.
- Exact queue/event-bus product selection.
- Exact search-engine product selection after measured workload requirements.
- Exact LLM/embedding providers subject to privacy, cost and latency requirements.
- Commercial settlement timing and seller payout policy.
- Retention periods and jurisdiction-specific privacy obligations.

These are controlled project decisions, not architectural gaps to be silently guessed by an implementation agent. Once approved, they must be recorded as configuration/ADR decisions and reflected consistently across API, database, infrastructure, testing and operational documents.